

SPECIFICATION STAGE / NO APPLICATION CODE

OpenBEXI Timeline 2.0

Implementation specification and generation prompt | Revision 2.0 | 12 September 2026

Project name: openbexi_timeline2.0

Reference repository: https://github.com/arcaj/openbexi_timeline

Source baseline: master at cf5d263853e550aab44d3d1959637c1e324b719e.

Current task: finalize this prompt only. Do not generate application code, install application dependencies, build the application, or deploy anything until the user explicitly requests implementation. Once implementation is authorized, execute the requirements and milestones below. JSON examples, API definitions, and acceptance scenarios are specifications, not authorization to start development.

This Markdown file is the editable source of the accompanying PDF. Keep their substantive content identical. The source audit is static: the legacy application was not built or run during this revision. Source-derived findings, proposed behavior, and future verification requirements are distinguished throughout.

CONTENTS / 23 REQUIREMENT AREAS

01. Mission, priorities, and storage boundary

02. Evidence-based legacy baseline

03. Architecture and authoritative state

04. Canonical records and relationships

05. Time semantics and legacy interpretation

06. JSON file layout and indexing

07. Durability, concurrency, and recovery

08. REST resource surface

09. API write, error, and retry semantics

10. Queries, filters, pagination, and export scope

11. Models, configuration, and saved views

12. Timeline experience and interaction
13. Table, selection, and shared state

14. Live updates and reconnection

15. Authorization, configuration safety, and operations

16. Import, migration, and preservation

17. Performance targets and supported scale

18. Acceptance scenarios: data and API

19. Acceptance scenarios: UI and operations

20. Delivery milestones after authorization

21. Decisions, exclusions, and prompt handoff

22. Worked contracts and traceability

23. Sources and reference standards

01. Mission, priorities, and storage boundary

Act as a senior application architect, reverse-engineering specialist, timeline developer, API designer, UX designer, and quality engineer. Design, and later implement when authorized, a modern successor to OpenBEXI Timeline. Preserve useful, source-verified temporal visualization capabilities; add a complete RESTful management API and a synchronized, fully usable tabular view.

REQ-01 - JSON files are the only persistent event/session store. Store all authoritative events, sessions, and child activities in ordinary UTF-8 .json files on a local filesystem. Do not introduce PostgreSQL, SQLite, MongoDB, Oracle, Elasticsearch, Redis, LevelDB, IndexedDB, an ORM-backed store, a database server, or another embedded database as an event/session store, index, cache, queue, or hidden dependency. A file-backed database is still prohibited. JSON Lines/NDJSON is not the canonical format: each persisted .json file must be one independently parseable JSON document.

Use JSON files for application-owned models, saved filters, UI configuration, views, audit records, job metadata, and recovery journals as a design default, keeping the deployment database-free. In-memory indexes and disposable JSON index snapshots are permitted. Static assets may retain their native formats; compressed backups may package JSON documents. Deployment environment variables and external secret files are not event stores.

REQ-02 - One writer, multiple clients. The initial supported deployment is one application server process owning one local data root, with concurrent browser/API clients. Enforce exclusive ownership across processes. Do not promise multi-server shared-disk writes, network filesystem safety, or horizontal write scaling. This constraint must appear in startup checks and deployment documentation.

REQ-03 - Product scope. The initial release includes event/session CRUD, nested activities, time navigation, configurable bands and grouping, visual models, shared filters, Timeline/Table/Split modes, durable settings, live synchronization, migration, backup/restore, and automated verification. Build a temporal visualization and editing tool. Automatic scheduling, dependency propagation, critical path, resource optimization, recurrence expansion, and external database/broker connectors are deferred unless separately requested. Preserve relevant legacy metadata without inventing those engines.

Priority order: explicit user constraints; data integrity and security; verified legacy meaning; mandatory API/UI workflows; accessibility and performance; optional enhancements. Never resolve a performance problem by violating JSON-only storage. Mark each requirement as mandatory or explicitly deferred; do not turn mandatory features into "where supported" options.

02. Evidence-based legacy baseline

REQ-04 - Reproducible discovery. Use the pinned commit above for comparison. Inventory tracked source, examples, configuration, tests, manifests, assets, and documentation. Record read coverage and exclusions. Generated/vendored files, IDE metadata, binary assets, and certificates need classification, not claims of behavioral verification. Inspect large datasets with structured parsers and report parse failures. Do not treat a class name, configuration example, dependency, icon, or Swagger entry as proof of a working feature.

The following findings informed this revision. File identifiers refer to the pinned-source references in section 23.

Evidence	Source-derived finding	Required treatment
S01-S03	JavaScript/Three.js frontend and Java/Tomcat backend are present.	Evaluate reuse and maintained replacements; do not assume dependencies are current or the build works.
S03-S04	UI logic uses an events collection, while the illustrative model uses a session envelope. Point/duration rendering depends on end-time parsing.	Write explicit import adapters and canonical kind rules; do not interchange envelopes blindly.
S03	Activities, overview bands, original-time graphics, shading, grouping, and camera controls exist in source.	Trace and preserve meaningful behavior with tests; distinguish time-region shading from time-zone conversion.
S05-S06	JSON sources, date-based paths, configurable rendering, and filter files exist alongside other connector examples.	Retain JSON ingestion and presentation semantics. Exclude all non-JSON storage/connectors from the new runtime.
S07-S08	JSON creation exists, but its construction/error handling are unsafe; update/remove return false and HTTP PUT/DELETE are not implemented as CRUD.	Implement new resource contracts; do not advertise legacy CRUD as complete.
S09	Swagger 2.0 describes one GET route and includes placeholders for other methods.	Replace it with a complete, tested API specification. Client-supplied permission strings are not authorization.
S10	The Dockerfile expects local JARs, lib, and node_modules.	Verify clean-build reproducibility later; source inspection is not a successful startup test.

During implementation discovery, run the original in an isolated environment when feasible, with throwaway data and local-only access. Record exact commands, screenshots, working workflows, failures, and benchmark conditions. Do not use bundled certificates or example credentials in the new deployment. If legacy execution is blocked, continue source-based compatibility work and label runtime comparisons unavailable.

Create a feature matrix with source location, evidence type, legacy meaning, preserve/replace/retire decision, new requirement ID, and acceptance test. Security defects, placeholder integrations, and broken writes are not compatibility obligations. Explain exclusions and never claim a complete runtime audit from static inspection.

03. Architecture and authoritative state

REQ-05 - Simple, modular architecture. Prefer one deployable application with a REST API and static frontend assets. Separate domain/time rules, JSON persistence, query/filter evaluation, authorization, API transport, shared client state, and rendering. Both views must invoke the same commands and consume the same canonical records. Files and browser state must not become competing authorities.

Evaluate the existing Java backend and JavaScript renderer before selecting frameworks. Record the choice, supported runtime versions, dependency maintenance, license compatibility, build reproducibility, and rendering tradeoffs in an architecture decision record. Do not force a rewrite solely for novelty. Preserve useful 2D/3D capabilities when supported by source evidence; choose the clearest accessible default. Avoid microservices, message brokers, database drivers, and unnecessary infrastructure.

Use established libraries for date/time handling, schema validation, HTTP, filesystem locking, and rendering where practical. Verify dependency behavior on the supported platforms rather than claiming generic portability. Lock versions and document a reproducible clean installation. Do not implement custom cryptography or evaluate arbitrary scripts from user configuration.

REQ-06 - Workspace boundary. A workspace owns records, sources, groups, schemas, visual models, filters, views, settings, a persisted generation UUID, and a monotonically increasing committed revision within that generation. Provide one default workspace; do not require an organization/tenant provisioning system. Keep IDs and permission checks workspace-scoped, with no accidental cross-workspace references or cache reuse. Use immutable startup configuration and request-local identity/query context; never put a client's mutable request state in a shared configuration object.

Restoring/replacing a workspace creates a fresh generation UUID. Bind ETags, query snapshots, cursors, change positions, and retry scopes to generation as well as revision. Mutations must declare the generation they observed through X-Workspace-Generation; an old generation returns 409 and requires refresh. Old retry results must not replay as current-generation mutations. Application-global resources use an equivalent root generation/revision.

Every durable mutation passes through validation, authorization, concurrency checks, persistence, audit, and post-commit publication. A successful API response means the documented commit protocol completed, not merely that an object changed in memory. Reads return a coherent committed revision. A failed save must not leave the timeline and table displaying a falsely confirmed value.

Use bounded work queues with overload responses. Long imports, exports, migrations, and backups run as tracked jobs in the same service, with JSON metadata and explicit progress/failure states. Their presence must not require Redis, Kafka, a queue service, or another database. Read-only static demonstration mode may be provided later, but it does not satisfy the server-backed release requirements.

04. Canonical records and relationships

REQ-07 - One record model with explicit kinds. An event is a point in time. A session is a duration record, optionally containing ordered child activities. An activity is a record linked to a parent session, not a second copy of its data. Sessions may exist without children. Authentication sessions are unrelated to timeline sessions and must use distinct terminology.

Field	Contract
id, workspaceId	Server-assigned opaque UUID and workspace identity. Immutable. Retain original IDs separately during import.
kind, title	event or session; required nonblank title, 1-500 characters. Kind is immutable after creation.
start, end	Required normalized start timestamp. Events have end: null; sessions have a finite end or explicit null for an ongoing session.
parentSessionId, order	Nullable parent ID and integer ordering key. Parent must be a session in the same workspace/source. Break order ties by ID.
sourceId, groupIds, tags	Required JSON source ID; arrays of stable group IDs and text tags. A default source/group is available.
data, render, extensions	Typed domain metadata, validated per-record presentation overrides, and preserved namespaced unknown metadata.
schemaId, schemaVersion	Optional pinned custom-field schema; core record fields always follow the application schema.
originalStart, originalEnd	Nullable baseline dates preserved independently of edited dates. PUT must include both explicitly; omission is invalid. PATCH changes them only through explicit allowed paths.
version, createdAt, updatedAt	Server-controlled positive integer record revision and audit timestamps; every successful mutation increments version.
createdBy, updatedBy, deletedAt	Server-resolved actors and nullable soft-deletion timestamp. Never accepted as client authority.

Validate unknown core keys as errors; preserve unknown legacy values under extensions.legacy. Declare built-in data fields explicitly: description/text/system/type/status are optional strings and priority is an optional finite number. Do not conflate data.text and description or coerce legacy strings without an import rule. Additional custom fields require schemaId and schemaVersion together; neither may appear alone. Remote schema resolution is disabled; references resolve only through approved local definitions with bounded validation complexity. Limit records to 256 KiB of serialized UTF-8, excluding separately managed assets; limit nesting to eight parent levels and reject cycles, self-parenting, dangling references, and duplicate IDs.

REQ-08 - Parent semantics. Moving/resizing a session does not implicitly move its children. Child times need not be contained within the parent; show an out-of-bounds indicator. A separate explicit "shift session and activities" command may use one bounded atomic batch. Do not derive the parent's dates silently from children.

Deleting a session with active children returns a conflict unless an explicit cascade operation lists the intended subtree and expected versions. Cascade and restore are atomic within the batch limit. A deleted parent cannot gain new children. Restore preserves IDs and increments versions; it fails if relationships would be invalid. Workspace/source/group/model deletion similarly rejects active references unless a documented reassignment or archival workflow is used.

05. Time semantics and legacy interpretation

REQ-09 - Deterministic time handling. Accept timestamp strings with an explicit numeric offset or Z; normalize stored instants to UTC with millisecond precision. Preserve an optional IANA display-zone identifier separately. Reject zone-less API timestamps, impossible dates, leap-second input, and precision beyond milliseconds rather than silently truncating. Support years 0001-9999 where the selected runtime can validate them consistently; document and test any narrower supported range before implementation acceptance. [N03]

For point events, end is null and duration is zero. For finite sessions, end must be greater than or equal to start. Equal endpoints represent a zero-duration session and retain session identity. For ongoing sessions, null end is explicitly meaningful only with kind: session. Elapsed duration is the maximum of zero and reference-instant minus start, never written back on every refresh. A future-start ongoing session displays a scheduled/not-yet-started cue, not negative elapsed time. Start/end edits do not automatically change domain status.

Use half-open query windows [from, to) and reject from >= to with 422. A positive-duration session matches if its start is before to and its end is after from; an ongoing session has no upper end. Point events and zero-duration sessions match when their start is inside the window. Include sessions that begin before the visible range but overlap it. Never rely on a start-date folder scan alone to find long-running sessions. Axis ticks use actual calendar boundaries in the selected display zone, never fixed 31-day months or 365-day years.

REQ-10 - No silent legacy reinterpretation. In the audited frontend, a missing/unparseable end follows the point-event rendering branch. During migration, missing/empty end therefore defaults to an event; it does not automatically become an ongoing session. A valid finite end maps to a session, including equal start/end. A nonempty invalid end is quarantined for review rather than accepted as a valid point. Preserve the original value and mapping decision. [S03-S04]

An empty-end legacy parent with genuine child activities conflicts with the new session-only parent rule. Default to quarantining that complete aggregate with all IDs and relationships. An explicitly selected import policy may create a provenance-marked finite session container spanning the point and validated child extents, retaining the original point as a child; if a finite extent cannot be determined, leave it quarantined. Never silently promote the original point into an ongoing session or flatten/drop its children.

Legacy Java-style date strings and different top-level envelopes require named adapters. An ambiguous timezone abbreviation or zone-less historical date needs an explicit import-time source-zone policy; otherwise reject that row with a reason. For DST gaps, reject nonexistent local times. For repeated local times, require the offset or an explicit earlier/later choice. Keep timezone display, elapsed duration, and calendar formatting consistent between views and API exports.

Preserve repeater, tolerance, conflict fields, original dates, and other legacy metadata without inventing recurrence or scheduling behavior. Legacy tolerance rendering does not establish a reliable time unit; retain it as raw metadata unless a documented mapping defines units. Provide fixtures for cross-midnight sessions, DST changes, overlaps, historical dates, ongoing sessions, and window boundaries.

06. JSON file layout and indexing

REQ-11 - Documented filesystem contract. Use a configured data root outside the publicly served asset tree. Default to a single canonical JSON document per record, distributed by an ID-derived directory prefix to avoid one oversized directory. A record's path must not depend on mutable dates, titles, usernames, filters, or source labels.

Relative area	Purpose
workspaces/{id}/records/{prefix}/{recordId}.json	Authoritative record document, including deletion metadata.
workspaces/{id}/config/{resource}/{id}.json	JSON sources, groups, models, schemas, views, and saved filters.
workspaces/{id}/settings/	Workspace and personal JSON settings with versions.
workspaces/{id}/journal/{transactionId}.json	Valid JSON recovery/commit envelope; includes before/after images, revision, audit, and retry metadata.
workspaces/{id}/audit/	Immutable JSON audit projections, recoverable from retained committed transactions.
workspaces/{id}/indexes/	Optional disposable JSON snapshots tagged with their workspace revision.
workspaces/{id}/jobs/	Validated JSON job state and import/export manifests.
backups/, quarantine/	Restricted verified backups and isolated suspect files with diagnostic manifests.

Root-level control JSON documents hold application defaults, principal/capability metadata, token hashes, workspace registry, generation, and root transaction/audit history. Raw token secrets never enter these files. Root-level mutations use a root transaction queue and the same recovery protocol; an operation requiring root and workspace locks always acquires root first, then workspace IDs in lexical order. Avoid unnecessary cross-workspace atomic operations.

The precise layout may change through a documented decision if benchmarks justify it, but not to a database, NDJSON canonical log, arbitrary executable format, or multiple independent authoritative copies. JSON parsing must reject duplicate object keys, non-finite numbers, invalid encoding, truncated documents, and unsupported future format versions. Persist explicit format/schema versions.

Build in-memory ID, time-overlap, group/source, parent, and supported custom-field indexes from validated committed records. Persisted index snapshots are caches, never authorities. Missing, corrupt, or stale indexes must be rebuildable without data loss. A suspect index triggers rebuilding or a clearly reported unavailable query; it must never cause silently incomplete search results.

Document directory limits, file counts, memory cost, index rebuild time, and record-size limits. Partitioning must preserve globally correct overlap queries and stable identities across date edits. Startup must acquire the process lock, recover transactions, validate storage, and load/rebuild indexes before becoming ready. Do not read or mutate arbitrary host paths supplied by clients.

07. Durability, concurrency, and recovery

REQ-12 - Explicit commit protocol. Use an OS-enforced exclusive process lock for the data root and serialize each workspace's mutations. A second writer must refuse startup; a stale filename alone is not a reliable lock. Reads may use the last committed immutable in-memory snapshot while a mutation is prepared; never expose a partially updated batch.

Document a recovery protocol before implementing it. The default is a JSON write-ahead transaction envelope with complete before/after images and checksums. Atomically install and flush the complete PREPARED envelope, including required directory metadata where supported, before changing any authoritative target. If preparation fails, do not touch targets. Stage replacement files on the same filesystem, flush contents/metadata using supported platform primitives, and atomically replace targets; never use delete-then-create as a commit mechanism. After all targets are durably installed, atomically persist the COMMITTED state, then publish the new in-memory revision and respond. Cleanup occurs after commit and must not invalidate recovery evidence.

A transaction lacking a durable commit marker is rolled back to its before state on restart; a committed transaction is replayed to its after state if necessary. Recover against a durable checkpoint and ordered sequence, checking before/after versions and checksums. Roll back incomplete transactions and replay committed transactions in revision order; never overwrite a newer committed version with an older after-image. Detect gaps or conflicting journal entries. Recovery is idempotent and completes before serving affected workspaces. Retain enough history to repair projections/indexes/audit. If classification is unsafe, fail closed and preserve evidence; never guess by file modification time.

REQ-13 - Failure scope. Guarantee coherent acknowledged writes under documented process-crash conditions. Validate disk-full, permission failure, file-sharing contention, interrupted writes, and crashes at each protocol boundary. Document separately what is supported for OS crash and power loss on Windows/NTFS and Linux/local filesystems; atomic rename alone is not proof of durable power-loss safety. Reject unsupported deployment filesystems rather than claiming untested guarantees.

Record data, record versions, workspace revision, audit payload, and idempotency result belong to the same logical transaction. Derived notifications and indexes occur only after commit. Never report success unless durable commit is established. Pre-commit or uncertain-commit I/O failures follow rollback/reconciliation; post-commit cleanup failures do not reverse an established result. Lost responses after commit are resolved by retry identity.

Complete rollback before accepting another mutation after a pre-commit failure; freeze the workspace if rollback fails. If an error leaves commit status uncertain, reconcile the durable journal before reporting a final outcome. A failed response or client disconnect never proves rollback. Once commit is established, report/replay its result even if later nonessential cleanup fails.

REQ-14 - Recovery operations. Detect malformed documents and unexpected out-of-band changes. Freeze affected workspace writes and report integrity errors; never overwrite an externally edited file silently. Supported external ingestion goes through staging/import. Direct editing of the live store is unsupported. A backup captures a consistent committed snapshot of records, tombstones, schemas/models, settings, root control state, identities/capabilities, retry/audit/replay history, and generations/revisions in a checksummed manifest. External secrets are excluded and their separate restore requirements documented. Limit the first release to validated restore into an isolated inactive root, then an offline root switch; no live in-place restore. Reject unsafe archive paths, oversized extraction, missing files, and checksum/format failures. Journal compaction requires a verified durable checkpoint and preserves declared recovery/history retention.

08. REST resource surface

REQ-15 - Complete API, independent of the UI. Implement `/api/v1`, with an OpenAPI 3.1.1 contract and JSON Schema Draft 2020-12 definitions. This is a deliberately pinned interoperability baseline, not a claim to use the newest specification. Contract changes must update examples, validators, and tests together. API documentation is mandatory. [N01-N02]

In the table below, B means `/api/v1/workspaces/{workspaceId}`. Resource URLs are illustrative normative route names, not application source code.

Endpoint family	Required methods and purpose
<code>/api/v1/workspaces, /{id}</code>	GET/POST collection; GET/PATCH/DELETE item. Deletion allowed only when empty; no implicit mass erase.
B/events, B/sessions	GET/POST collection; GET/PUT/PATCH/DELETE on <code>/ {id}</code> . Typed projections of the same canonical store.
B/records, B/records/{id}	GET combined collection/item. Authorized <code>includeDeleted=true</code> queries and item reads expose tombstones/current ETags for restore.
B/records/query	POST a structured read-only query with filters, scope, sorting, cursor, and optional totals.
B/query-snapshots/{id}	DELETE an owned snapshot handle when no longer needed; changes no canonical record.
B/records/batch	POST a bounded atomic mixed-record command with expected versions.
B/records/{id}/restore	POST restore a tombstoned record with concurrency preconditions.
B/sources, B/groups	Collection GET/POST and item GET/PUT/PATCH/DELETE. Sources are JSON-only logical namespaces.
B/schemas, B/models	CRUD for schema/visual-model metadata; versioned definitions and reference-safe retirement.
B/filters, B/views	CRUD for named filters and saved timeline/table layouts with ownership/visibility.
B/settings, B/preferences/me	GET/PUT/PATCH persisted settings; DELETE resets allowed overrides.
<code>/api/v1/settings</code>	Admin GET/PUT/PATCH/DELETE for mutable application defaults; DELETE resets overrides.
<code>/api/v1/principals, /{id}</code>	Admin GET/POST collection; GET/PATCH/DELETE item. DELETE disables identity and revokes its tokens.
<code>/api/v1/tokens, /{id}</code>	Authorized GET/POST collection and DELETE revocation; one-time token secret on creation.
B/changes, B/stream	GET committed change pages or authenticated SSE stream.
B/imports, B/exports, B/jobs/{id}	POST validated jobs; GET authorized status/results; explicit pre-commit cancellation.
B/audit, B/backups	Authorized GET audit; admin POST/GET backups and validated restore workflow.

Define exact subroutes for version history, publish/archive, backup restore, and job cancellation in the OpenAPI design milestone. Provide all request/response fields, examples, permissions, pagination, limits, and error cases before handler implementation. Expose `/health/live`, `/health/ready`, and `/api/v1/capabilities`; the latter reports supported modes, configured limits, API/storage versions, and read-only/recovery state without secrets.

New JSON source definitions never accept an arbitrary absolute data path or a different storage type. Legacy paths belong only to administrator-controlled migration mappings. Configuration CRUD does not authorize changing server secrets, process identity, filesystem roots, or transport security through the browser.

09. API write, error, and retry semantics

REQ-16 - Predictable HTTP behavior. GET is read-only. Collection creation returns 201 with Location; structured POST queries return 200; synchronous batch/restore commands return 200; accepted asynchronous jobs return 202 with a status Location. PUT replaces the complete declared mutable representation, including explicit baseline-date fields; PATCH uses JSON Patch with an allowlist and explicit null semantics. Arrays follow the patch contract, not undocumented merges. Record DELETE soft-deletes with 204; settings DELETE resets overrides; definition deletion follows reference/archive rules. Invalid resource kinds on typed routes return 404. [N04, N07]

Every mutable resource returns a strong ETag derived from its generation and server revision.

PUT/PATCH/DELETE/restore require If-Match: missing preconditions return 428, stale versions return 412, and business/reference conflicts return 409. Batch items carry equivalent expected revisions. Check versions inside the serialized commit boundary, not only when the request arrives. Ordinary reads exclude tombstones and return 404 for deleted item IDs; an authorized trash/includeDeleted read returns the tombstone and current ETag. Restore requires edit/restore capability. Retain schema/model references for restorable deleted records. Do not implement silent last-write-wins.

Require an Idempotency-Key for record creation, batches, import commits, and restore operations. Scope keys to principal, generation, workspace, method, and route; hash semantic request content including applicable preconditions and persist the result. After authentication, authorization, and generation validation, check retry identity before current resource preconditions: a completed identical retry replays its original result even when the original If-Match is now stale. A changed request conflicts. Persist pending identity before mutation and recover it after restart. Retain completed keys for at least 24 hours and publish that limit. A pending identical retry returns a documented 202 with status Location; do not duplicate work or replay currently unauthorized sensitive output.

REQ-17 - Structured failures. Use application/problem+json with type, title, status, detail, instance, stable application code, request ID, and field errors. Do not include secrets or absolute internal paths. Distinguish malformed JSON (400), missing authentication (401), denied permission (403), absent/inaccessible resources (404 under a documented disclosure policy), conflict (409), invalid fields (422), limit violations (413/429), stale preconditions (412/428), and unavailable storage (503). Include Retry-After when applicable. [N05]

Bound atomic batches to 500 affected records and 8 MiB of serialized UTF-8 request data, including cascades. Also cap the complete serialized before/after recovery envelope at 32 MiB; reject excessive expansion before preparation, even for a tiny patch. Validate authorization, schemas, uniqueness, references, and expected versions for every item before any write. A single failed item rejects the whole batch with item-level diagnostics. Larger imports use explicitly non-atomic chunks with a manifest of committed/failed IDs; never describe them as globally transactional.

Define cleanly whether cancellation happened before or after commit. After commit, cancellation cannot erase the mutation; return its committed result. Client disconnects do not imply rollback. Document request/body limits, rate limits, timeout handling, unsupported media types, and retry safety. Provide examples for create, stale edit, ongoing-session close, cascade rejection, batch rollback, and retry after lost response.

10. Queries, filters, pagination, and export scope

REQ-18 - One filter language. Define a versioned JSON expression tree shared by saved filters, API queries, timeline selection, and table results. Support nested and, or, not; typed equality/inequality, ordered comparisons, in, text contains, and field existence. Time overlap is a first-class predicate with section 05 semantics. Allow only declared core/custom fields; no SQL fragments, executable JavaScript, filesystem paths, or arbitrary regular expressions.

Specify string matching as Unicode-normalized, case-insensitive contains by default; provide an explicit case-sensitive option. Define null versus missing separately, typed numeric/date comparison, array membership, and stable sorting. Missing values satisfy only explicit missing/existence tests unless the operator's contract states otherwise. Reject mixed-type ordering and unknown fields rather than coercing them unpredictably. Bound expression depth to eight, total predicates to 100, and list operands to 100 values.

Cross-schema queries resolve custom fields through an explicit schema/version scope. A field declared in that scope but absent from a record is missing. Incompatible types under the same field name require a narrower schema predicate; they are not coerced. Evaluate each record independently. Optional authorized ancestors are returned as separately flagged context, excluded from matching counts, exports, and implicit bulk selection. Expanding a parent does not broaden the filter or reveal unauthorized children.

REQ-19 - Stable result sets. Use cursor pagination with deterministic requested sort plus ID as the tie-breaker. Default page size is 100, maximum 1,000. Bind opaque/tamper-resistant cursors to generation, workspace, authorized scope, filter hash, sort, and an immutable query snapshot. Retain snapshots for five minutes, with a configurable per-principal limit of four and an initial global retained-memory budget of 256 MiB. Share immutable backing data where possible; reject new snapshots with 429 when capacity cannot preserve active promises. Re-check access on every page. Expiry/restart/restore returns an explicit snapshot-expired conflict, never silent omissions. Later writes, preferences, or job progress do not invalidate an active snapshot.

Clients release superseded snapshot handles after no view/page/export uses them; reference-count shared backing snapshots and invalidate released handles, preventing live refresh from exhausting its allowance. A permission-scope change invalidates affected snapshots in full: deny further pages and require a newly authorized snapshot. Never silently remove rows while retaining old totals/aggregates. Revoked authentication still returns 401/403; a changed but valid scope returns an explicit refresh-required conflict.

Both views either show the same pinned snapshot or refresh together to a new one. In paginated browsing mode, incoming changes show a "new changes available" state without silently changing the snapshot's rows/counts; refresh preserves selection and the nearest stable anchor. In live mode, refresh/reconcile immediately. A local successful edit refreshes both views to a new snapshot. Do not mix old snapshot rows with new totals. Exports also pin a committed snapshot. Demonstrate reaching page ten during five writes/second without endless restarts.

Return items, next cursor, workspace revision, applied scope, and exact total when requested. Counts, group aggregates, and density summaries must use the same predicate/revision as the record result. Never display an approximate or loaded-row count as an exact filtered total. For expensive totals, make the separate/pending state explicit.

The viewport is a rendering/query-loading window, not automatically a user's data filter. Default table results reflect the full saved filter; a clearly named visible-range mode intentionally adds a time predicate. Timeline loaded counts may therefore differ from full table totals, but counts for identical predicates/revisions must match. Exports specify selected IDs, all filtered records, or visible range; they must not silently export only the loaded page.

Offer lossless canonical JSON import/export and a documented legacy JSON adapter. CSV is a tabular convenience export with explicit nested-field flattening, timezone, escaping, and spreadsheet-formula protections; it is not a full-fidelity configuration or data backup.

11. Models, configuration, and saved views

REQ-20 - Separate concepts currently called models. A data schema defines typed custom fields and validation. A visual model defines bands, lanes, grouping, styles, time scales, overview relationships, camera mode, labels, and legends. A saved view references models, filters, columns, sorting, timezone, and viewport preferences. None is an AI model. The legacy `models/regular_timeline.json` is presentation configuration and must not be treated as a database schema. [S11]

Provide JSON-backed management for data schemas, visual models, saved filters, sources, groups, views, and configurable UI settings. Each has identity, schema/format version, resource revision, name, ownership, and explicit visibility. Validate import/export using the same server validators as CRUD. Duplication generates new IDs and preserves a traceable relationship to the original.

REQ-21 - Version and reference safety. Referenced schema/model definitions are immutable definition versions, distinct from mutable resource metadata revisions. Editing produces a new definition version; existing records/views remain pinned until an explicit migration/update. Compatible optional-field additions may use a documented opt-in upgrade; renames, type changes, and required-field additions require impact preview. Publishing a model version is atomic. Reject deletion of versions referenced by active or restorable deleted records/views; permit archival without breaking them. New definitions must not silently invalidate saved filters or columns.

Define settings precedence as application defaults, workspace defaults, visual-model values, saved-view overrides, then personal overrides, with ephemeral interaction state last. Permissions and hard server limits always constrain the result. Per-record rendering overrides permitted style fields, but cannot bypass accessibility rules or authorization. Scalars replace; maps merge by allowed key; arrays replace unless a schema explicitly uses stable keyed members. Distinguish clearing an override from setting a nullable value. Provide a reset-to-inherited action and an effective-settings inspection API.

REQ-22 - Customization coverage. Support band sizes/order, overview synchronization, grouping and collapsed state, event/session colors and approved icons, conditional style rules, label fields, font size bounds, time scales/formats, display timezone, current-time indicator, original-time overlays, shading/regions, table column visibility/order/width, typed sorting, and saved query selection. Sanitize URLs/assets and validate contrast-related limits. No arbitrary HTML, JavaScript, remote code modules, or unbounded CSS injection.

Represent shaded regions as versioned visual-model entries with stable IDs, finite start/end, label, band/group scope, visibility, and validated style. They are annotations, excluded from record counts and record bulk actions; document whether a view filter hides their scoped groups. Migrate legacy zone records through this map and preserve provenance. Search/selection highlights are transient client styling and never mutate stored render overrides. Distinguish legacy grouping named `sortBy` from actual table sort.

Support personal and workspace-shared views/filters. Owners manage personal resources; administrators manage shared defaults. An editor may publish shared resources only with explicit workspace capability. Opening a saved view restores its intended state; switching views does not discard unsaved edits silently. Startup-only settings such as data root, bind address, and secrets are documented separately as read-only effective metadata where safe.

12. Timeline experience and interaction

REQ-23 - A usable operational workspace. Open directly into populated timeline data or a clear empty workspace, with a compact command bar, view switch, source/group controls, shared filters, and an event inspector. Prioritize scanning and repeated work. Preserve multiple bands, overview navigation, useful grouping, nested activities, legends, point/session distinctions, original-time overlays, and relevant camera behavior identified in the feature matrix.

Provide pan, zoom, fit-to-data/filter, jump to date, jump to selected record, and optional follow-now. Explain the actual range through axis labels. Preserve navigation on resize, sorting, filter edits, and view changes. Fetch overlapping records with a modest buffer and cancel obsolete requests. Avoid background refreshes that unexpectedly jump the user's viewport.

REQ-24 - Real editing workflows. Provide create, inspect, duplicate, edit, move, resize, delete, restore, and batch actions subject to permissions. Point events can move but cannot resize into sessions implicitly. Finite sessions can move/resize with documented snapping; ongoing sessions have an explicit close action and no fabricated end handle. Dragging a parent does not move children without a separate command. Precise forms and keyboard controls must be available for every mutation.

Show draft, saving, saved, conflict, and failure states. Prefer optimistic previews with clear pending status; reconcile with server-normalized records on success and restore the prior confirmed state on rejection. Preserve the user's draft during conflicts and show changed fields; require an explicit reapply against a new revision. Undo/redo creates compensating version-checked writes, never rewrites history or silently overwrites another user's work. Define its bounded session history and deletion/restore behavior.

Resolve overlapping/dense events with deterministic lanes, readable hit targets, clustering/density summaries at extreme zoom, and a path to inspect underlying records. Aggregates must be labeled and must not hide missing data silently. Provide empty, loading, malformed-data, disconnected, read-only, recovering, and permission-denied states that preserve useful content.

Keyboard navigation, focus management, screen-reader record access, touch alternatives, reduced motion, and non-color status cues are mandatory. Target applicable WCAG 2.2 AA criteria with automated and manual checks. On narrow screens, use one primary view plus accessible details and deliberate horizontal scrolling; do not shrink a desktop timeline into unreadable controls. Canvas/WebGL content needs an accessible equivalent through the table and inspector. [N06]

13. Table, selection, and shared state

REQ-25 - A working table, not a report screenshot. Provide Timeline, Table, and resizable Split modes. Derive table columns from core fields and pinned schemas: title, kind, start, end/ongoing state, derived duration, source, group, parent, status, tags, custom fields, and optional ID/version. Format missing/not-applicable values distinctly. Provide column visibility/order/resize, typed sort, filters, details, inline edits, and authorized bulk actions.

Concern	Required shared behavior
Identity	A row and timeline item reference the same workspace/record ID and server version.
Selection	Selection survives view switching and pagination. Filter-hidden IDs are indicated; revoked/deleted IDs are marked unavailable and sensitive cached content is removed.
Navigation	Selecting a row highlights its item when loaded; an explicit reveal action loads and navigates to its time. Selection alone need not jump the viewport.
Filters/counts	Identical predicates at an identical revision produce identical IDs/counts. Distinguish all-filtered totals from viewport-loaded counts.
Edits	Both views issue the same validated command and show its pending/result/conflict states. No page reload is needed.
Sorting	Table sort does not unexpectedly reorder timeline lanes or reset the viewport.
Parent/children	Tree/group presentation preserves relationships; selecting a session is not implicit selection of every child.
Refresh	Merge by ID/version, remove tombstones, discard stale responses, and preserve drafts/selection.

Virtualize large result sets without making inaccessible rows the only path to a record. Support keyboard focus across virtualization boundaries. Keep row dimensions stable with long titles/custom fields and expose full values through accessible details.

Explicitly distinguish "select this page", individual selected IDs, and "all matching records". A bulk action is bound to the displayed query/revision, previews its exact scope, and revalidates on commit. If it exceeds the 500-record atomic limit, require an explicit job/chunk workflow or reject it with actionable detail. Never mutate an unbounded result set because a header checkbox was ambiguous.

Provide copy/export actions with the scope rules from section 10. A saved view stores column and grouping preferences through the API. Browser memory may cache UI state, but confirmed event/session persistence always comes from server JSON files; reload must demonstrate durable results.

14. Live updates and reconnection

REQ-26 - Publish committed changes. Provide authenticated Server-Sent Events with a polling fallback using B/changes. Each transaction envelope carries workspace ID, generation, monotonically increasing revision/sequence, transaction ID, and a bounded changes array of operation, resource type/ID, and version. Clients apply the complete authorized group atomically before recomputing results, or receive an invalidation envelope requiring requery if the payload is too large. Transport delivery is at least once; clients deduplicate. Include configuration/filter/model changes as well as records.

Use a snapshot-then-resume handshake: query results identify a committed revision, then the client consumes changes after that revision. Subscribe/replay must cover the interval between snapshot and subscription. Never subscribe only to records currently matching a filter without a membership-change strategy; records can enter or leave the result set when edited. Notify the authorized workspace and re-evaluate affected queries safely.

Retain replay history of at least 10,000 transactions or 24 hours, whichever requires more history. Bound disk growth through capacity planning, quotas, compaction of older eligible history, and refusal/backpressure on new writes when retention cannot be met; do not silently shorten the promised window. If a change cursor is expired, invalid, or from a restored generation, return an explicit full-resynchronization signal. A fresh snapshot replaces stale cached state while preserving drafts. Follow the pinned-versus-live display rules in section 10.

REQ-27 - Failure behavior. Reconnect with bounded exponential backoff and jitter, support heartbeat/idle detection, and show connection/save state. Do not enqueue offline writes automatically in the initial release. Users may retain an in-memory draft while disconnected, but saving requires reconnection and revision validation. Polling fallback has the same ordering and authorization rules as SSE.

Authenticate the stream with the same opaque bearer token through a header-capable SSE client; do not add a separate login/session store. Do not place bearer tokens in URLs. Permission revocation closes/restricts active streams, invalidates affected caches, and prevents unauthorized resource disclosure. A principal permitted to see only part of a workspace must not receive IDs or payloads from the rest.

On server restart, reconstruct replay positions from durable committed JSON history. Backpressure must be bounded: slow clients disconnect and resume or resynchronize rather than growing an unlimited queue. Delete/restore ordering, duplicate messages, missed revisions, stale query responses, and simultaneous local/remote edits all require integration tests.

15. Authorization, configuration safety, and operations

REQ-28 - Server-enforced permissions. Provide viewer, editor, and administrator roles with explicit workspace/resource capabilities. Viewers read authorized records/views; editors create/update/delete records and manage their personal views; administrators manage sources, shared configuration, identities, backups, and migrations. Audit/export access is explicit and must not leak hidden fields. Never trust a role, username, userAccess, or ownership field submitted by a client as proof of permission.

The first release uses opaque high-entropy API tokens with server-side hash verification and an administrator bootstrap secret supplied through environment/secret configuration. Principal metadata, capability grants, expiry/revocation state, and token hashes persist as protected root JSON documents. Show new token secrets once; token listing exposes metadata only. Principals may list/revoke their own tokens; admins create identities and issue scoped tokens with explicit expiry (default 30 days). Rotation creates a replacement then revokes the old token. Bootstrap works only for an uninitialized root and must not leave a permanent universal token. Keep browser tokens in memory, not URLs or durable browser storage; logout clears them. Disabling a principal/revoking a token immediately blocks further requests and streams. No external identity service or cookie-login system is required.

Restored token records remain disabled. Issue fresh credentials through a documented local administrator recovery procedure before exposing a restored service. Restoring an older root must never reactivate a revoked token or reopen network bootstrap. Preserve principal IDs/ownership metadata while re-establishing current access. Backup/restore API jobs prepare and validate inactive roots; activation remains an offline operator action.

REQ-29 - Validate real boundaries. Apply the same permissions to API, import/export, batch commands, audit, and streaming. Constrain CORS/origins and request sizes; rate-limit expensive reads and writes. Safely render user labels, descriptions, links, icons, imported metadata, and error details. Prevent mass assignment, prototype pollution, path traversal, symlink/reparse-point escape, arbitrary file access, and script/HTML injection. Resolve all storage paths under validated internal roots using server-generated names.

Provide a sanitized effective-configuration endpoint and validation preview. Runtime UI changes cannot alter storage roots, TLS keys, trusted origins, executable paths, or identity bootstrap secrets. Reject unsupported storage types at schema validation and startup. Keep JSON data directories and backups out of static hosting and deployment images.

REQ-30 - Practical operations. Provide clean startup/shutdown, readiness after recovery, restricted logs, correlation IDs, and metrics for reads/writes, queue depth, conflicts, file count, index rebuild, disk space, stream clients, and backup age. Do not expose record payloads or credentials by default in logs. Back up data and configuration together with versions/checksums; document encrypted storage/backup options without embedding secrets.

Support Windows local development and a Linux container with a persistent local volume and one writer. Document platform-specific guarantees and actual tests. Startup errors must identify unavailable storage, permission problems, unsupported format, or competing writer clearly. Restore/purge are administrator operations with previews; normal DELETE remains recoverable. Never copy bundled legacy private keys into a new deployment.

16. Import, migration, and preservation

REQ-31 - Explicit compatibility map. Supply adapters for actual legacy JSON formats discovered at the pinned commit: events collections, the illustrative session envelope where applicable, flat records, and genuine nested activities. Do not persist synthetic frontend singleton wrappers as duplicate activities. Preserve stable legacy identifiers through an import mapping table; canonical new IDs are UUIDs. Detect collisions per source/workspace and report them before commit.

Map `data.title`, start/end/original dates, domain metadata, per-record render values, grouping, and nested activity relationships. Keep raw unknown metadata under namespaced extensions. Migrate visual models, overview bands, source definitions, saved filter names/grouping, and allowed UI preferences separately from records. Record every unsupported setting or unsafe value in a migration report. [S03-S06, S11]

Legacy include/exclude/filter expressions require parsing into the new typed expression tree. If equivalence cannot be established, mark the filter as needing review and preserve its original text; do not silently broaden it to ALL or execute legacy expressions as code. Placeholder database definitions, connector credentials, and permission strings do not become new runtime configuration. External database migration, if later requested, must arrive as an offline JSON export; the new application does not connect to those stores.

REQ-32 - Dry run first. Analyze input without mutating it; report file/record counts, checksums, IDs, inferred kinds, relationships, date conversions, invalid JSON, unsupported fields, and collisions. Invalid syntax is rejected with location information. An optional repair step must produce a separate reviewed artifact and an explicit change log; never repair a source file silently. Imports use strict canonical validation after transformation.

Assign an import ID and stable row mapping so retries after restart cannot duplicate records, including sources without IDs. Normal imports generate canonical IDs with a stable mapping and rewrite internal references; duplication also generates new IDs. Explicit canonical restore preserves IDs/server metadata into an empty isolated target and assigns a new workspace generation. Lossless round-trip claims apply to this restore mode; normal import preserves payload meaning with documented identity remapping and new audit fields. Default collision policy is reject; updates require explicit mapping and expected versions. Commit small imports atomically; large imports publish chunk results and a resumable manifest. Conditional rollback affects only untouched imported versions and reports records subsequently edited.

Inventory date-derived descriptor sidecars and merge/reference their content by stable legacy ID so changing dates cannot orphan descriptions. Preserve `data.text` separately from description and resolve conflicting namespace fields explicitly. Translate legacy source/group names to stable IDs; never fan one create operation out to every configured source. [S12-S13]

Keep originals and verified backups. Compare normalized before/after IDs, timestamps, relationships, metadata, render intent, and filter result sets using representative fixtures. Require JSON export/import round trips and restore drills. Compatibility means preserving meaningful data and supported behavior; it does not mean reproducing invalid JSON, unescaped HTML, silent write failures, or incomplete connectors.

17. Performance targets and supported scale

REQ-33 - Measurable starting targets. The following are proposed release targets, not measured legacy results. Freeze the benchmark environment and targets before implementation tuning. Any justified change must be recorded with its effect; do not quietly weaken targets after a failure. JSON-only storage remains mandatory regardless of scale.

Reference environment: one 4-core/16-GiB machine with local SSD, one server process, production frontend build, local network, current supported desktop Chromium, and 1440x900 viewport. Record exact OS, filesystem, CPU, browser/runtime versions, fixture checksum, network latency, cache state, and repeat count. Test Windows/NTFS and Linux/local container volume separately for storage behavior.

Scenario	Initial acceptance target
Typical fixture	100,000 records, 100 groups, nested activities, dense overlap and mixed historical/ongoing data; average serialized record ≤ 2 KiB.
Cold readiness	Recovery-free startup and index rebuild ≤ 30 seconds for the typical fixture. Crash recovery measured separately by journal size.
Visible data query	Warm p95 ≤ 300 ms for a 1,000-record page, supported indexed predicates, no optional full-result export.
Single-record save	End-to-end local API durable commit p95 ≤ 300 ms at five aggregate writes/second.
First useful workspace	≤ 3 seconds from navigation to interactive visible data after the server is ready.
Pan/zoom and table scroll	p95 frame time ≤ 33 ms during a fixed 10-second trace with $\leq 2,000$ visible primitives; no repeated > 200 ms main-thread stalls.
Shared updates	p95 ≤ 1 second from commit to a second client's confirmed live-mode display; pinned browsing receives an update notice within the same budget.
Concurrency/memory	20 connected clients, five writers within the aggregate rate; server RSS ≤ 2 GiB for the typical fixture; no sustained growth after repeated cycles.

Also test 1,000-record small and 1,000,000-record stress fixtures, explicitly labeling the stress tier unsupported until measured. Include long sessions overlapping many dates, long labels, sparse custom fields, deletions, corrupt indexes, and update bursts. A directory of one million files is a cost to measure, not a scalability claim.

Profile parsing, index construction, overlap filtering, disk flushes, serialization, client state, and rendering. Use viewport culling, level-of-detail aggregation, table virtualization, incremental updates, bounded caches, and workers when measurements justify them. Exact complex-filter totals and large exports may run as jobs; disclose their latency. Never fabricate before/after performance or conceal data omission behind a faster rendering result.

18. Acceptance scenarios: data and API

REQ-34 - Requirements map to executable tests. The following are minimum future acceptance scenarios. During the present prompt-only task, they are specifications and are not claimed as executed application tests.

ID	Scenario and required outcome
A01	Create an event and a finite session through the API; reload/restart; the same IDs, values, versions, and relationships remain in valid JSON files.
A02	Import empty-end legacy records; they become points, while explicit new ongoing sessions remain durations. Nonempty malformed dates are rejected/quarantined.
A03	Query [10:00,11:00); a 09:00-12:00 session matches, a session ending at 10:00 does not, and a point at 11:00 does not.
A04	Exercise DST gap/fold, offset conversion, equal endpoints, original dates, historical dates, and sub-millisecond input; results follow section 05 exactly.
A05	Two clients edit version 7; one succeeds as version 8, the other receives 412 and retains its draft. No update is silently lost.
A06	Kill the process at every transaction boundary; recovery yields the old complete state without a commit marker or the new complete state with one, never a mixed batch.
A07	Inject disk-full, permission and replacement failures; no false success is returned, acknowledged data remains recoverable, and diagnostics identify the failed operation.
A08	Retry creation/batch after a committed write loses its HTTP reply and after restart; one mutation exists and the stored result is replayed. Changed retry payload returns 409.
A09	Submit a batch with one invalid/stale/unauthorized item; no item is committed and field/item errors identify the cause.
A10	Delete a parent with children; default deletion conflicts. Explicit bounded cascade and restore commit atomically and preserve IDs.
A11	Remove/corrupt cached JSON indexes; rebuild returns the same query IDs/counts, including sessions crossing date boundaries.
A12	Start a second writer or modify a live file externally; the service refuses competing writes or freezes the affected workspace without silent overwrite.
A13	Reach page ten under continuous writes using one snapshot without duplicate/missing rows; changed query, expired snapshot, or restore generation fails explicitly.
A14	Attempt cross-workspace reads/writes, forged roles, traversal, unsafe imports, and stream access after revocation; access is denied consistently.
A15	Inspect installation, runtime services, filesystem artifacts, and dependencies; events/sessions use only ordinary JSON files and memory, with no hidden database/broker.

Use deterministic fixtures and controllable clocks. Verify stored documents with a strict independent JSON parser, inspect record versions and transaction state, and assert API status/body contracts. Storage tests must use real temporary filesystems as well as fault injection; mocks alone do not prove atomic replacement or process locking.

19. Acceptance scenarios: UI and operations

ID	Scenario and required outcome
A16	Create/edit/delete/restore from either view; both views reconcile the same canonical records and confirm durable state after reload.
A17	Switch Timeline/Table/Split; filter, selection, timezone, and intended viewport survive. Identical predicates/revisions yield equal IDs/counts.
A18	Sort the table and navigate to an unloaded selected row; timeline lanes remain stable and explicit reveal loads the correct event.
A19	Drag/resize with snapping and keyboard alternatives; stale writes conflict; moving a parent does not silently move children.
A20	Save personal and shared views/models/filters, restart, and reset overrides; precedence, ownership, versions, and effective settings match section 11.
A21	Try an incompatible schema change or delete a referenced model/group/source; validation blocks it until an explicit migration/reassignment.
A22	Drop, duplicate, reorder, and expire live notifications; clients deduplicate/requery/resynchronize without losing a committed edit or stale deletion.
A23	Import representative valid/invalid legacy fixtures, resume after interruption, export, and re-import; mappings, unknown metadata, and counts are audited without duplicates.
A24	Create a backup during writes and restore it into an isolated root; all documents/configuration/checksums agree at one committed revision.
A25	Exercise long labels, dense overlaps, empty/error/loading/recovery states, narrow screens, keyboard-only use, focus and screen-reader access. No inaccessible canvas-only mutation remains.
A26	Run section 17 benchmarks and repeated open/filter/close/reconnect cycles; publish measured results, resource cleanup checks, and any target misses.
A27	Follow README from a clean checkout; build/start/test commands work with a local JSON directory and no database installation.
A28	Export selected/all-filtered/visible-range records; the declared scope is honored across pagination, with safe CSV formatting and lossless canonical JSON.
A29	Match a child whose parent is filtered out; show authorized ancestor context separately, excluded from totals/export/bulk selection. Validate mixed-schema field/null rules.
A30	Preserve overview/band synchronization, grouping versus sort, original-time overlays, approved styles, tolerance metadata, and declared camera behavior through migration/edit/reload.
A31	Save/reload/export/import shaded regions, and process a point parent with genuine activities; annotation counts and aggregate quarantine/mapping follow their explicit contracts.
A32	Restore a backup then retry an old-generation edit; reject it. Revoke credentials and run concurrent users with different filters; verify request isolation and zero cache/stream leakage.

REQ-35 - Honest verification. Add unit tests for time, validation, filter semantics, references, and state transitions; integration tests for JSON recovery, APIs and synchronization; browser tests for real workflows; and visual/accessibility checks for representative screens. Test actual Chrome/Edge/Firefox where available and document Safari/WebKit/mobile coverage precisely. An automated accessibility scan alone is not WCAG conformance evidence.

Report commands, environment, test totals, outcomes, artifacts, benchmark distributions, and manual checks. Distinguish passed, failed, skipped, blocked, and not run. Fix discovered defects and rerun affected tests. Do not claim zero defects or complete performance compliance merely because a screenshot looks polished or a narrow test passed.

Release acceptance requires every mandatory requirement and A01-A32 to pass in the declared supported scope. Extend A22 to a multi-record cascade/batch and prove neither view briefly exposes a partial transaction. Unresolved mandatory failures mean the implementation is incomplete, even if a demonstration can run. Optional deferred features must remain clearly identified in documentation and must not appear as working controls.

20. Delivery milestones after authorization

REQ-36 - Incremental implementation. This section activates only after the user explicitly requests application implementation. Work in `openbexi_timeline2.0`, preserve the legacy baseline, and maintain decision/requirement records as evidence changes. Do not ask again about routine choices already resolved by this specification; document any genuinely blocking conflict before dependent work.

Milestone	Deliverable and gate
M0: audit and contracts	Complete source/feature inventory, runtime observations if possible, architecture decisions, storage protocol, schemas, OpenAPI and representative examples. No unresolved contradiction with JSON-only persistence.
M1: JSON vertical workflow	One real event/session workflow from API to durable JSON to Timeline and Table. Include restart, invalid input, and stale-write tests before broad UI expansion.
M2: reliable management	Full CRUD, relationships, bounded batches, recovery, audit, authorization, imports, backup and restoration. Pass core storage/API scenarios.
M3: configurable workspace	Visual models, schemas, shared filters, saved views, settings, synchronized edits and full table workflows. Validate persisted customization and references.
M4: live experience	SSE/polling, reconnect, navigation, dense rendering, accessible editing, responsive layouts and visual regressions.
M5: release verification	Migration round trips, fault tests, clean setup, all acceptance scenarios, benchmark results, support limits and final documentation.

Each implementation milestone M1-M5 must remain runnable and have a short evidence report; M0 produces reviewed contracts and discovery evidence. A mock-data screenshot is not completion of a storage/API milestone. Do not postpone correctness until after visual polish. Fix failed mandatory gates before claiming the next dependent milestone is complete.

REQ-37 - Documentation deliverables. Author project documentation in Markdown. Keep README practical: purpose, supported platforms/versions, installation, configuration, startup, local URL, sample JSON data, authentication bootstrap, backup warning, test commands, and links to detailed guides. Detailed documents cover source audit, architecture decisions, data/time schema, JSON storage/recovery, API, filtering, configuration precedence, user workflows, development, testing, performance, migration, backup/restore, deployment, troubleshooting, and requirements traceability.

During implementation, provide native OpenAPI and JSON Schema artifacts as machine-readable specifications, linked from Markdown. Preserve licenses/attribution, maintain a changelog, and document schema/API/storage version compatibility. Provide reproducible fixtures and test reports. No placeholder routes, fake persistence, unfinished required controls, example secrets, or database installation instructions belong in the delivered release.

21. Decisions, exclusions, and prompt handoff

The following are deliberate design defaults for this revision, not claims about the legacy implementation. They remove ambiguity while keeping implementation choices evidence-driven.

Decision	Rationale / boundary
JSON-only event/session persistence	Explicit user constraint; not negotiable through framework choice or scaling optimization.
JSON-backed configuration/audit/jobs	A database-free deployment with one recovery/backup approach; non-JSON static assets and secret configuration remain allowed.
Single server writer / local disk	Practical cross-file transaction and locking scope. Multiple API users remain supported.
Unified typed records / session children	Retains point/duration distinctions and genuine nested activities without duplicating records.
Ongoing sessions are explicit	Prevents empty legacy end fields from silently changing meaning.
Soft delete / bounded atomic batches	Recovery and predictable failure semantics without unbounded distributed transactions.
REST + SSE / polling fallback	Complete management plus committed updates without a required broker.
Versioned schemas/models and typed filters	Controlled customization and shared timeline/table meaning.
Framework/renderer choice deferred to M0	Must follow source evidence, accessibility, maintenance, and measured behavior rather than an arbitrary stack mandate.
No automatic scheduling/recurrence engine	Preserve metadata/visual behavior; add these product capabilities only under a later explicit request.

The source-only audit leaves legacy runtime behavior, installation viability, real throughput, and exact renderer tradeoffs unverified. These are M0/benchmark tasks, not invented facts or reasons to weaken mandatory requirements. Runtime/library versions and platform-specific locking/flush mechanisms must be verified when implementation begins. No mandatory product choice currently requires a new user answer to finish this prompt.

Prompt-only completion gate. Deliver this revised Markdown and a matching, visually checked PDF; retain the original brief; link source evidence; identify assumptions and exclusions; and confirm that no application code was produced. Remove instructions that could trigger implementation during this refinement task. Do not describe the specification as proof that the future application is already correct.

Later implementation completion gate. Deliver runnable code, verified JSON storage, complete API/configuration management, synchronized timeline/table workflows, migration/backup tools, test results, benchmarks, and documentation only after implementation authorization. Evaluate completion against requirements and acceptance scenarios, not against the presence of files alone.

22. Worked contracts and traceability

These are specification examples, not application code. M0 must encode them in the native schemas/OpenAPI and add complete request/response examples before handlers are written.

Example	Required interpretation
Point creation	Title "Telemetry received", kind event, start 2026-09-12T14:00:00.000Z, end null, default JSON source. A successful create returns 201, Location, generated UUID, version 1, generation, and ETag.
Duration creation	Title "Observation window", kind session, start 2026-09-12T14:00:00.000Z, end 2026-09-12T14:30:00.000Z. Duration is 30 minutes, calculated rather than separately editable.
Ongoing/parented record	The same session with null end is explicitly ongoing. An activity is a separately stored point/session with that session's ID as parent; it is not duplicated in a stored child array.
Filter expression	An internal node has op equal to and/or and args children; not has one arg. A leaf has op, an allowed JSON Pointer field, and typed value/values when needed. Example meaning: status equals STARTED AND priority is at least 2, restricted to the declared schema scope.
Query request	Contains filter version/tree, source/schema scope, optional overlap window, ordered sort fields/directions, page limit, optional cursor, and includeTotal. Returns 200 with items, snapshot ID/revision/generation, nextCursor and declared totals. Reject cursor/query disagreement.
Stale update	The client submits an allowed patch with a valid generation and an old ETag. Return 412 problem details with code record_version_conflict, request ID and a safe current-version reference; do not apply any patch operation.
Replay	The original idempotency key and identical request replay the prior result within retention, after authorization/generation checks and before stale original If-Match rejection.

The built-in filter leaf operator names are eq, ne, lt, lte, gt, gte, in, contains, and exists. exists accepts a boolean and distinguishes missing from explicit null. Empty and/or groups and in lists are rejected. A top-level null filter means all authorized records; clearing a saved filter is an explicit action. Typed equality to null matches explicit null only. These rules must be identical in API, UI, migration tests and exports.

Requirement coverage	Minimum proof
REQ-01-06	Source/dependency/architecture review; A01, A12, A15, A27, A32.
REQ-07-14	Schemas and storage/recovery design; A01-A12, A23-A24, A31-A32.
REQ-15-19	OpenAPI and query contracts; A03-A05, A08-A10, A13-A14, A28-A29.
REQ-20-27	Configuration and shared UI/live state; A16-A22, A25, A29-A31.
REQ-28-32	Auth/operations/migration; A14-A15, A23-A24, A27, A31-A32.
REQ-33-37	Recorded benchmarks, all acceptance scenarios, runnable milestones, clean setup and documentation audit.

Expand this map to per-requirement implementation paths and executable test IDs during implementation; scenario IDs alone do not prove coverage of every clause.

23. Sources and reference standards

Source observations were made from a shallow local clone on 12 September 2026 at commit `cf5d263853e550aab44d3d1959637c1e324b719e`. Links below are pinned to that commit. This is a targeted static audit, not a claim that all legacy code paths were executed. See `docs/legacy-source-audit.md` for evidence locations, limitations, and changes from the original seven-page brief.

- S01 - [Legacy package manifest](#): frontend dependency declarations.
- S02 - [Legacy Maven manifest](#): Java level, server and connector dependencies.
- S03 - [Frontend source](#): rendering, activity initialization, navigation, grouping and editing paths.
- S04 - [Illustrative event/session model](#): legacy envelope, dates and metadata.
- S05 - [Default source configuration](#): JSON path templates and other connector examples deliberately excluded.
- S06 - [Default saved filter settings](#): source, presentation and named-filter configuration.
- S07 - [JSON file manager](#): read/filter/write behavior and update/delete stubs.
- S08 - [AJAX servlet](#): action-based dispatch and method coverage.
- S09 - [Legacy Swagger](#): incomplete advertised API.
- S10 - [Legacy Dockerfile](#): deployment assumptions requiring later validation.
- S11 - [Regular timeline visual model](#): parameters, bands, scales, colors and grouping.
- S12 - [Descriptor sidecar manager](#): date-derived descriptor files requiring ID-based migration.
- S13 - [Historical sample data](#): historical timestamps, text fields and absent IDs.

Normative reference standards are chosen for specific contracts, not as claims about the newest available versions:

- N01 - [OpenAPI 3.1.1](#): API contract representation.
- N02 - [JSON Schema Draft 2020-12](#): validation dialect.
- N03 - [RFC 3339](#): timestamp foundation; this specification deliberately restricts accepted precision and leap seconds.
- N04 - [RFC 9110](#): HTTP semantics and conditional requests.
- N05 - [RFC 9457](#): problem-details response format.
- N06 - [WCAG 2.2](#): applicable AA accessibility criteria.
- N07 - [RFC 6902](#): JSON Patch operations with application path restrictions.

End of specification and generation prompt. Application implementation remains a separate, explicitly authorized task.